Python Programming

Part 4 — Functions, Decorators, Lambda, Higher-Order Functions & Generators

Class Notes

Table of Contents

27. User-Defined Functions

A function is a reusable block of code that performs a specific task. A **user-defined function** is one that you create yourself using the `def` keyword, as opposed to Python's built-in functions like `print()` or `len()`. Functions let you avoid repeating the same code, and they break a big program into smaller, manageable, testable pieces.

Syntax: `def function_name(parameters):` followed by an indented code block.

Advantages

- Avoids repeating the same code multiple times
- Makes code easier to test, debug, and reuse across projects

Limitations

- Too many tiny functions can make code harder to follow if not organized well
- Function calls have a small performance overhead compared to inline code

Applications

Where this is used in Data Science:

- Wrapping a data-cleaning routine into one function, reused across multiple datasets
- Building a reusable function to calculate a metric (e.g. accuracy, RMSE) for every model tested

EXAMPLE 1 — SIMPLE GREETING FUNCTION

```
def greet(name):
    print(f"Hello, {name}!")

greet("Riya")
greet("Aman")
```

Output:

```
Hello, Riya! Hello, Aman!
```

EXAMPLE 2 — FUNCTION THAT CALCULATES AND RETURNS A VALUE

```
def calculate_average(marks):
    total = sum(marks)
    return total / len(marks)

result = calculate_average([80, 90, 70])
print(result)
```

Output:

```
80.0
```

28. Function Arguments — Positional, *args, **kwargs, Order

Arguments are the values you pass into a function when calling it. Python supports several ways of passing arguments, each suited to different situations.

Type	Meaning
Positional arguments	Values matched to parameters strictly by their position/order
*args	Collects any number of extra positional arguments into a tuple
**kwargs	Collects any number of extra keyword arguments into a dictionary

Order that matters

When combining argument types in a function definition, they must follow this fixed order: **positional arguments** → ***args** → **default arguments** → ****kwargs**. Breaking this order causes a SyntaxError.

Applications

> **Where this is used:**
>
> - `*args` is used when a function should accept a variable number of numeric values, e.g. calculating the sum of any number of columns
> - `**kwargs` is used heavily in libraries like Matplotlib/Pandas, where functions accept many optional settings, e.g. `plt.plot(x, y, color="red", linewidth=2)`

EXAMPLE 1 — POSITIONAL ARGUMENTS VS *ARGS

```
def add(a, b):
    return a + b

print(add(5, 10))

def add_all(*args):
    return sum(args)

print(add_all(1, 2, 3, 4, 5))
```

Output:

```
15 15
```

EXAMPLE 2 — **KWARGS AND CORRECT ORDER OF ALL ARGUMENT TYPES

```
def student_info(name, *args, grade="A", **kwargs):
    print("Name:", name)
    print("Extra subjects:", args)
    print("Grade:", grade)
    print("Other details:", kwargs)

student_info("Aman", "Maths", "Science", grade="A+", city="Pune", age=20)
```

Output:

```
Name: Aman Extra subjects: ('Maths', 'Science') Grade: A+ Other details:
{'city': 'Pune', 'age': 20}
```

💼 **Common Interview Question:** What is the difference between *args and **kwargs, and why does their order matter in a function definition?

Note: `*args` collects extra unnamed values as a tuple; `**kwargs` collects extra named (key=value) values as a dictionary. Python needs positional arguments identified first, hence the fixed order.

29. Global and Local Variables

A **local variable** is created inside a function and only exists while that function is running — it cannot be accessed outside it. A **global variable** is created outside all functions, at the main level of the program, and can be accessed from anywhere, including inside functions (but not modified directly without special handling).

Comparison

Aspect	Local Variable	Global Variable
Defined	Inside a function	Outside all functions
Accessible from	Only within that function	Anywhere in the program
Lifetime	Destroyed when function ends	Exists for the whole program run
Modifying inside a function	Normal assignment works	Requires the `global` keyword

Advantages

- Local variables keep functions independent and avoid accidental interference
- Global variables are useful for shared settings/constants used across the program

Limitations

- Overusing global variables makes code harder to debug — any function could change them
- Modifying a global variable inside a function without the `global` keyword causes errors or unexpected behavior

Applications

> **Where this matters in Data Science:**
>
> - Configuration values (like a dataset file path or a random seed) are often kept global so every function uses the same setting
> - Local variables keep intermediate calculations inside a function from accidentally leaking into or overwriting other parts of a script

EXAMPLE 1 — LOCAL VARIABLE SCOPE

```
def calculate():
    result = 100   # local variable
    print(result)

calculate()
print(result)   # this line will cause an error
```

> **Output:**
>
> 100 NameError: name 'result' is not defined

EXAMPLE 2 — MODIFYING A GLOBAL VARIABLE USING THE GLOBAL KEYWORD

```
counter = 0

def increment():
    global counter
    counter += 1

increment()
increment()
print(counter)
```

> **Output:**
>
> 2

> 💼 **Common Interview Question:** What happens if you try to modify a global variable inside a function without using the global keyword?

30. return vs print

`print()` only displays a value on the screen — it does not give that value back to the program. `return` sends a value back from the function to wherever it was called, so that value can be stored, reused, or passed into another function.

Comparison

Aspect	print()	return
Purpose	Displays output on screen	Sends a value back to the caller
Can the value be reused?	No, it's just shown, not stored	Yes, it can be stored in a variable and reused
Stops the function?	No	Yes, return immediately ends the function
Value outside the function	Not accessible	Accessible via the variable it was assigned to

Applications

Why this matters:

- A function meant to calculate a value (like an average or a prediction) should use `return` so the result can be used later in the code — e.g. saved to a file, plotted, or passed to another function
- `print()` is fine for quickly checking/debugging values, but is not useful when the result needs to be reused programmatically

EXAMPLE 1 — PRINT() CANNOT BE REUSED, RETURN CAN

```
def add_print(a, b):
    print(a + b)

def add_return(a, b):
    return a + b

add_print(5, 10)          # just displays 15

result = add_return(5, 10) # actually stores 15
print(result * 2)          # reuse the value
```

Output:

```
15 30
```

EXAMPLE 2 — CHAINING FUNCTIONS ONLY WORKS WITH RETURN

```
def square(x):
    return x * x

def cube(x):
    return x * x * x

value = square(3)
final = cube(value)   # using the returned value in another function
print(final)
```

Output:

```
729
```

💼 **Common Interview Question:** If a function only uses print() and no return, what does calling that function and storing it in a variable give you?

Note: A function without `return` gives back `None` by default. So `result = add_print(5, 10)` would store `None` in `result`, even though "15" was printed on screen.

31. Docstrings

A docstring is a special string written as the very first line inside a function, class, or module, used to describe what it does. It is written using triple quotes `"""..."""` and can be accessed later using the `help()` function or `.__doc__` attribute — this is different from a regular `#` comment, which is only for developers reading the raw code.

Applications

Where this is used:

- Documenting data-processing functions so other team members understand what a function expects and returns, without reading the full code
- Auto-generating documentation for a data science project/library using tools that read docstrings

EXAMPLE 1 — BASIC DOCSTRING AND ACCESSING IT

```
def add(a, b):
    """Returns the sum of two numbers a and b."""
    return a + b

print(add.__doc__)
```

Output:

```
Returns the sum of two numbers a and b.
```

EXAMPLE 2 — MULTI-LINE DOCSTRING WITH HELP()

```
def calculate_average(marks):
    """
    Calculate the average of a list of marks.

    Parameters:
    marks (list): list of numeric marks

    Returns:
    float: average value
    """
    return sum(marks) / len(marks)

help(calculate_average)
```

Output:

```
Help on function calculate_average in module __main__: calculate_average(marks) Calculate the average of a list of marks. Parameters: marks (list): list of numeric marks Returns: float: average value
```

Tip: Well-written docstrings are especially valuable in data science projects, where functions are often shared and reused across a team or in published libraries.

32. Decorators — Timer & Logger Decorator

A decorator is a function that wraps around another function to add extra behavior, without changing the original function's code. Decorators use the `@decorator_name` syntax placed just above a function definition. They are used to add reusable functionality — like timing, logging, or access checks — to multiple functions cleanly.

Applications

Where this is used in Data Science:

- **Timer decorator:** measuring how long a data-processing step or model training function takes to run
- **Logger decorator:** automatically recording every time a function is called, and with what arguments, without adding logging code inside every single function

Advantages

- Adds reusable functionality (timing, logging, validation) without duplicating code
- Keeps the original function clean and focused only on its main task

Limitations

- Can make debugging slightly harder, since the actual function is wrapped inside another
- Syntax can be confusing for beginners at first

EXAMPLE 1 — TIMER DECORATOR

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end - start:.4f} seconds")
        return result
    return wrapper

@timer
def process_data():
    total = sum(range(1000000))
    return total

process_data()
```

> **Output:**
> process_data took 0.0123 seconds

EXAMPLE 2 — LOGGER DECORATOR

```
def logger(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__} with args={args}, kwargs={kwargs}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} returned {result}")
        return result
    return wrapper

@logger
def add(a, b):
    return a + b

add(5, 10)
```

> **Output:**
> Calling add with args=(5, 10), kwargs={} add returned 15

> 💼 **Common Interview Question:** Explain how a decorator works internally — what does the @ symbol actually do?

> **Note:** `@timer` above a function is just shorthand for writing `process_data = timer(process_data)` — the decorator wraps the original function inside another function that adds extra behavior around it.

33. Lambda Functions

A lambda function is a small, anonymous (unnamed) function defined in a single line using the `lambda` keyword, instead of a full `def` block. It is used for short, simple operations, often passed directly as an argument to another function.

Syntax: `lambda arguments: expression`

Comparison — lambda vs normal function

Aspect	Lambda function	Normal function (def)
Name	Anonymous (can be assigned a name, but not required)	Always has a name
Length	Single expression only, one line	Can contain multiple lines/statements
Best used for	Short, throwaway logic, often passed as an argument	Reusable, more complex logic

Advantages

- Very compact for simple one-line operations
- Convenient to pass directly into functions like sort, map, filter

Limitations

- Cannot contain multiple statements or complex logic
- Overusing lambda for complex logic hurts readability — a normal function is clearer

Applications

Where this is used in Data Science:

- Sorting a list of records by a custom key, e.g. sorting students by marks
- Used constantly inside Pandas with `.apply(lambda x: ...)` to transform a column in one line

EXAMPLE 1 — BASIC LAMBDA VS NORMAL FUNCTION

```
square_normal = lambda x: x * x
print(square_normal(5))

def square_def(x):
    return x * x
print(square_def(5))
```

> **Output:**
> 25 25

EXAMPLE 2 — USING LAMBDA AS A SORT KEY

```
students = [("Aman", 78), ("Riya", 92), ("Karan", 65)]
sorted_students = sorted(students, key=lambda student: student[1])
print(sorted_students)
```

> **Output:**
> [('Karan', 65), ('Aman', 78), ('Riya', 92)]

> 💼 **Common Interview Question:** Why can't a lambda function contain multiple statements like a normal function?

34. List Comprehension with Lambda

List comprehension and lambda can be combined — a lambda function can be called inside a list comprehension to apply custom logic to every item while building a new list, all in one compact line.

Applications

Where this is used:

- Applying a custom transformation to every value in a dataset column, without writing a full for loop
- Creating a list of processed/derived values using a reusable lambda rule

EXAMPLE 1 — APPLYING A LAMBDA INSIDE LIST COMPREHENSION

```
square = lambda x: x * x
numbers = [1, 2, 3, 4, 5]
squared_list = [square(n) for n in numbers]
print(squared_list)
```

Output:

```
[1, 4, 9, 16, 25]
```

EXAMPLE 2 — CONDITIONAL TRANSFORMATION USING LAMBDA IN LIST COMPREHENSION

```
label = lambda marks: "Pass" if marks >= 40 else "Fail"
marks_list = [55, 30, 78, 20]
results = [label(m) for m in marks_list]
print(results)
```

Output:

```
['Pass', 'Fail', 'Pass', 'Fail']
```

> **Tip:** This pattern is very close to what happens internally when you write `df["column"].apply(lambda x: ...)` in Pandas — the lambda logic is applied to every item in a collection.

35. map, filter, reduce

These are three built-in "higher-order functions" — functions that take another function as input and apply it across a collection. They are commonly combined with lambda for short, one-line data transformations.

Function	What it does
map(func, iterable)	Applies a function to every item and returns the transformed results
filter(func, iterable)	Keeps only the items for which the function returns True
reduce(func, iterable)	Combines all items into a single value by repeatedly applying the function (needs `from functools import reduce`)

Comparison — map/filter/reduce vs list comprehension

Aspect	map/filter/reduce	List comprehension
Readability	Can feel less readable, especially reduce	Usually more readable and "Pythonic"
Best used when	Applying an existing function across data quickly	Writing custom logic directly inline
Output type	map/filter return an iterator (needs list() to view)	Directly returns a list

Applications

> **Where this is used in Data Science:**
>
> - `map()` : applying a transformation function (like unit conversion) across an entire dataset column
> - `filter()` : keeping only rows/values that meet a certain condition, e.g. sales above a threshold
> - `reduce()` : combining values into one summary result, e.g. computing a running total or product

EXAMPLE 1 — MAP() AND FILTER()

```
numbers = [1, 2, 3, 4, 5]

squared = list(map(lambda x: x ** 2, numbers))
print(squared)

even = list(filter(lambda x: x % 2 == 0, numbers))
print(even)
```

> **Output:**
>
> `[1, 4, 9, 16, 25] [2, 4]`

EXAMPLE 2 — REDUCE()

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
total = reduce(lambda a, b: a + b, numbers)
print(total)

product = reduce(lambda a, b: a * b, numbers)
print(product)
```

> **Output:**
>
> `15 120`

> 💼 **Common Interview Question:** How would you find the maximum value in a list using reduce()?

36. Generators and yield

A generator is a special type of function that produces a sequence of values one at a time, instead of computing and storing them all in memory at once. Instead of `return`, a generator uses `yield`, which pauses the function and remembers exactly where it left off, resuming from that point the next time a value is requested.

Comparison — generator vs normal function returning a list

Aspect	Normal function (returns a list)	Generator (yield)
Memory usage	Stores the entire result in memory at once	Produces one value at a time — very memory efficient
Best for	Small datasets that fit comfortably in memory	Very large or infinite sequences
Values available	All at once	Only one at a time, in order

Advantages

- Extremely memory efficient for large datasets, since values are generated on demand
- Can represent infinite sequences, which a regular list cannot

Limitations

- Values can only be looped through once — you cannot go back or access by index
- Slightly less intuitive to read/debug than a normal function

Applications

Where this is used in Data Science:

- Reading and processing very large files or datasets line-by-line/row-by-row without loading everything into memory
- Feeding data in small batches into a machine learning model during training (used internally by data loaders)

EXAMPLE 1 — BASIC GENERATOR USING YIELD

```
def count_up_to(n):
    count = 1
    while count <= n:
        yield count
        count += 1

for num in count_up_to(5):
    print(num)
```

Output:

```
1 2 3 4 5
```

EXAMPLE 2 — GENERATOR FOR READING LARGE DATA IN CHUNKS

```
def even_numbers(limit):
    for i in range(limit):
        if i % 2 == 0:
            yield i

gen = even_numbers(10)
print(next(gen))
print(next(gen))
print(list(gen))
```

Output:

```
0 2 [4, 6, 8]
```

💼 **Common Interview Question:** What is the difference between return and yield, and why are generators memory efficient?

37. Recursive Functions

A recursive function is a function that calls itself in order to solve a problem by breaking it down into smaller versions of the same problem. Every recursive function needs a **base case** — a condition that stops the recursion — otherwise it will call itself forever and crash with a stack overflow error.

Advantages

- Naturally elegant solution for problems that are defined in terms of smaller versions of themselves (trees, nested structures)
- Often results in shorter, cleaner code than an equivalent loop

Limitations

- Uses more memory than loops, since each call is kept on the call stack
- Missing or incorrect base case causes infinite recursion and a crash
- Can be slower than an iterative (loop-based) solution for large inputs

Applications

Where this is used in Data Science:

- Traversing nested/tree-like data structures, such as JSON data or decision tree models
- Certain mathematical algorithms (like some sorting and searching algorithms) are naturally recursive

EXAMPLE 1 — SUM OF NUMBERS FROM 1 TO N, USING RECURSION

```
def sum_recursive(n):
    if n == 0:          # base case
        return 0
    return n + sum_recursive(n - 1)

print(sum_recursive(5))
```

Output:

```
15
```

EXAMPLE 2 — RECURSION TO REVERSE A STRING

```
def reverse_string(s):
    if len(s) == 0:      # base case
        return s
    return reverse_string(s[1:]) + s[0]

print(reverse_string("python"))
```

Output:

nohtyp

💼 **Common Interview Question:** What happens if a recursive function has no base case?

38. Factorial Example (Recursion vs Loop)

Factorial is the classic example used to teach recursion. The factorial of a number n (written n!) is the product of all whole numbers from 1 to n. It can be solved either using a loop (iterative approach) or recursion — comparing the two side by side makes the idea of recursion much clearer.

> 💼 **Common Interview Question:** Write a program to calculate factorial using both recursion and a loop, and explain the difference.

EXAMPLE 1 — FACTORIAL USING A LOOP

```
def factorial_loop(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

print(factorial_loop(5))
```

> **Output:**
> 120

EXAMPLE 2 — FACTORIAL USING RECURSION

```
def factorial_recursive(n):
    if n == 0 or n == 1:   # base case
        return 1
    return n * factorial_recursive(n - 1)

print(factorial_recursive(5))
```

> **Output:**
> 120

Comparison

Aspect	Loop version	Recursive version
Memory usage	Low — single running total	Higher — each call stays on the call stack until it returns
Readability	Straightforward and familiar	Elegant, closely matches the mathematical definition
Risk	No risk of stack overflow	Very large n can cause a stack overflow (recursion limit)

> **Tip:** In interviews, being able to explain factorial using both approaches — and explain the base case in the recursive version — shows a solid grasp of recursion, which is one of the most commonly tested basic concepts.